



Вот мой код для визуализированной игры камень ножницы бумага:

```
import tkinter as tk
from random import randint
import time
from PIL import ImageTk, Image as PILImage
from main import bot_turn, move_result, possible_move
```

Функция вызываемая при нажатии на виджет. Вызывает ход игры

```
def on_click(event):
    player_move = event.widget.name
    choosed = label_hide(root,
        list_obj_labels=list_obj_labels,
        name_label_to_show=player_move
    )
    bot_move = bot_chose_animation(bot_label=bot_label)
    attack(choosed)
    attack(bot_label)

    # Функция полета применятся к тому виджету который "победил",
    # если ничья все возвращаются
    # Добавить чистую функцию возвращения для бота установка знака вопроса!
    print(f'player move: {player_move}, bot move: {bot_move}')
    move_result(player_move, bot_move)
```

Чтобы возвращать виджету бота картинку вопроса, пропишем ему поле с дефолтной картинкой

так же вынесем создание и подрезку фото в отдельную функцию

```
def get_and_crop_img_obj(img_path, width=100, height=100):
    img = PILImage.open(img_path)
    scaled = img.resize((width, height))
    img_obj = ImageTk.PhotoImage(scaled)
    return img_obj
```

```

def create_widget(root,
    img_path,
    x, y,
    name: str,
    clickable=True,
    default_img_path=None,
    default_unvisible=False,
    width=100,
    height=100):
    # добавим еще поле с дефолтной картинкой и добавим параметр в функцию
    """ root - основное окно / img_path - путь к картинке
        x, y - координаты для позиционирования картинки
        name - имя виджета, для отслеживания на что мы нажали """

    photo_obj = get_and_crop_img_obj(img_path, width=width, height=height)
    label = tk.Label(root, image=photo_obj)
    if default_img_path is not None:
        default_photo_obj = get_and_crop_img_obj(default_img_path)
        label.default_image = default_photo_obj
    label.image = photo_obj
    label.name = name
    label.x = x # !!
    label.y = y # !!!
    label.place(x=x, y=y)
    if default_unvisible:
        label.place_forget()
    if clickable:
        label.configure(cursor="hand2")
        label.bind("<Button-1>", on_click)
    return label

def bot_chose_animation(bot_label, total_delay=700):
    """ Анимация выбора хода ботом. """
    image_paths = ('scissors.png', 'stone.png', 'paper.png')
    images = []
    for path in image_paths:
        img = PILImage.open(path).resize((100, 100))
        images.append(ImageTk.PhotoImage(img))
    bot_move_number = randint(9, 11)
    current_step = 0
    bot_chose = image_paths[bot_move_number % 3].split('.')[0]

    def step():
        nonlocal current_step
        idx = current_step % 3
        image = images[idx]

```

```

bot_label.configure(image=image)
bot_label.image = image
current_step += 1
if current_step <= bot_move_number: # Включая финал
    step_delay = total_delay // bot_move_number
    root.after(step_delay, step)

step()
return bot_chose

def label_hide(root, list_obj_labels, name_label_to_show, delay=2000):
    """ Скрываем остальные лейблы предметов кроме того что выбрал игрок """
    unhided = None
    for item in list_obj_labels:
        if item.name == '?':
            continue
        if item.name != name_label_to_show:
            item.place_forget()
        else:
            unhided = item
    # через delay_ms снова показываем все
    root.after(delay, all_label_show, list_obj_labels) # obj.after(delay, func_name, *args)
    return unhided

def all_label_show(list_obj_labels):
    """ Показываем все виджеты возвращаем дефолтную картинку бот вилжету """
    for item in list_obj_labels:
        if item.name == '?':
            item.configure(image=item.default_image)
            item.image = item.default_image
            item.place(x=item.x, y=item.y)

def attack(item,
            delay=300,
            frames=10,
            root_width=800,
            root_height=300,
            start_delay=850):
    """ Анимация атаки предмета. Летит в центр! """
    current_x = item_x = item.winfo_x()
    current_y = item_y = item.winfo_y()
    item_width = item.winfo_width()
    item_height = item.winfo_height()
    path_x = (item_x - root_width // 2) + item_width // 2
    path_y = (item_y - root_height // 2) + item_height // 2
    step_x = -path_x // frames
    step_y = -path_y // frames

```

```

step_delay = delay // frames
cross_middle = False
def step():
    nonlocal path_x, path_y, current_x, current_y, cross_middle
    if path_x * (path_x + step_x) <= 0 or path_y * (path_y + step_y) <= 0:
        cross_middle = True
    path_x += step_x
    path_y += step_y
    current_x += step_x
    current_y += step_y
    item.place(x=current_x, y=current_y)
    if not cross_middle:
        root.after(step_delay, step)
root.after(start_delay, step)

def attack_to_side(item,
    who_whins: str):
    """ Отправляет виджет к стороне противника """
    if who_whins == 'bot':
        pass # летим в сторону игрока пока не долетим до края
    elif who_whins == 'player':
        pass # летим в сторону бота

def explosion_hide(item):
    item.place_forget()

def explosion_animation(item, root, delay=1000):
    item.place(x=item.x, y=item.y)
    root.after(delay, item.place_forget)

def bot_return():
    """ Возвращает виджет бота на место """
    pass

```

Инициализация окна

```

root = tk.Tk()
root.geometry("800x300")
root.title("Игра Камень/Ножницы/Бумага")

bot_label = create_widget(root=root,
    img_path="question-mark.png",
    x=100,
    y=80,
    name="?",
    clickable=False,
    default_img_path="question-mark.png")

```

```

scissors = create_widget(root=root,
    img_path="scissors.png",
    x=680,
    y=80,
    name="scissors")
stone = create_widget(root=root,
    img_path="stone.png",
    x=580,
    y=80,
    name="stone")
paper = create_widget(root=root,
    img_path="paper.png",
    x=480,
    y=80,
    name="paper")
explosion = create_widget(root,
    img_path='explosion.png',
    x = 300,
    y = 50,
    name='explosion',
    default_unvisible=True,
    width=200,
    height=200)

list_obj_labels = [scissors, stone, paper, bot_label]

root.mainloop()

```

основная анимация запускается в функции `on_click()`:

Как мне организовать код так чтобы анимации выполнялись друг за другом, я хочу после вызовов :

```

attack(choosed)
    attack(bot_label)

```

запускать другие анимированные действия. Но ! проблема в том что если я например вызову какую то функцию `root.after(3000, new_anim)` после `attack(bot_label)` то какую бы я задержку не ставил в последующем `root.after()` анимация выполняется раньше чем завершатся все предыдущие анимации. Я не понимаю как работает `obj.after()`/

Проблема в том что

Отличный вопрос и очень типичная проблема при работе с `tkinter` и анимациями: `root.after` и `label.after` не ждут, они просто **планируют** выполнение через заданное число миллисекунд, независимо от того, дошли ли до этого места уже предыдущие планы.

Как работает `after`

- `root.after(delay_ms, func)` — ставит задачу в очередь на выполнение **через** `delay_ms` **миллисекунд**, не блокируя интерфейс.
- Но если ты пишешь:

```
attack(choosed)
attack(bot_label)
root.after(3000, new_anim)
```

то `attack` запускают `root.after` внутри себя, а `root.after(3000, new_anim)` запускается **немедленно как план**, а не по завершению анимаций.

Поэтому `new_anim` начинается раньше, чем завершатся анимации из `attack`.

Как правильно «ждать» окончания анимации

Нужно, чтобы функция, запускающая анимацию, **последовательно** вызывала следующую анимацию, а не полагалась на абсолютные задержки.

Вот идея, которую можно применить к твоему коду:

1. Сделать анимации «цепочками»

Вместо того чтобы в `on_click` городить такие линии:

```
attack(choosed)
attack(bot_label)
root.after(3000, new_anim) # ✗ по времени, не по логике
```

придумай единую функцию, которая запускает анимации **по цепочке**:

```
def run_animation_sequence(root, player_item, bot_move_str, delay=1000):
    # Функция, которая запускает одну анимацию и затем следующую
    def next_step():
        # Запускаем анимацию игрока
        attack(player_item, lambda: next_step2())

    def next_step2():
        # Запускаем анимацию бота
        attack(bot_label, lambda: next_step3())

    def next_step3():
        # Показываем взрыв
        explosion_animation(explosion, root, delay=delay)
        # После взрыва — можно вернуть всё
        root.after(delay, reset_items)

    # Запускаем первую анимацию
    next_step()
```

Но для этого нужно, чтобы attack принимал **callback** — функцию, которая запустится по завершению анимации.

2. Изменить attack так, чтобы он вызывал callback в конце

Сейчас ты делаешь так:

```
def step():
    nonlocal path_x, path_y, current_x, current_y, cross_middle
    ...
    if not cross_middle:
        root.after(step_delay, step)
```

Нужно добавить параметр `on_finish=None` и вызвать `on_finish()` в конце анимации:

```
def attack(item,
            delay=300,
            frames=10,
            root_width=800,
            root_height=300,
            start_delay=850,
            on_finish=None): # <<< новый аргумент
    current_x = item_x = item.winfo_x()
    current_y = item_y = item.winfo_y()
    item_width = item.winfo_width()
    item_height = item.winfo_height()
    path_x = (item_x - root_width // 2) + item_width // 2
    path_y = (item_y - root_height // 2) + item_height // 2
    step_x = -path_x // frames
    step_y = -path_y // frames
    step_delay = delay // frames
    cross_middle = False

    def step():
        nonlocal path_x, path_y, current_x, current_y, cross_middle
        if path_x * (path_x + step_x) <= 0 or path_y * (path_y + step_y) <= 0:
            cross_MIDDLE = True
        path_x += step_x
        path_y += step_y
        current_x += step_x
        current_y += step_y
        item.place(x=current_x, y=current_y)
        if not cross_middle:
            root.after(step_delay, step)
        else:
            # Анимация закончилась
            if on_finish:
                root.after(0, on_finish) # можно root.after(0, ...) или просто root.

    root.after(start_delay, step)
```

И тогда можно использовать цепочки:

```
def attack_sequence():
    # Анимация игрока → потом бота → потом взрыв
    attack(choosed, on_finish=lambda: attack(bot_label, on_finish=lambda: explosion_
```

3. В on_click уже не использовать абсолютный root.after по времени

Старую «плохую» часть:

```
attack(choosed)
attack(bot_label)
root.after(3000, new_anim)
```

заменить на:

```
def on_click(event):
    player_move = event.widget.name
    choosed = label_hide(root, list_obj_labels=list_obj_labels, name_label_to_show=p

    # Анимация выбора бота
    bot_move = bot_chose_animation(bot_label) # ← эту функцию тоже нужно сделать цеп

    # Цепочка анимаций
    attack(
        choosed,
        on_finish=lambda: attack(
            bot_label,
            on_finish=lambda: explosion_animation(
                explosion,
                root,
                delay=1000,
                on_finish=reset_items # можно добавить callback в explosion_animatio
            )
        )
    )
```

4. При необходимости сделать bot_chose_animation тоже цепочкой

Сейчас она завершается, но ты не можешь привязать after к концу её кадров. Тогда можно:

- Внутри bot_chose_animation в конце анимации вызывать on_finish().

```
def bot_chose_animation(bot_label, total_delay=700, on_finish=None):
    # ... как сейчас, но в конце
    def step():
        nonlocal current_step
        idx = current_step % 3
        image = images[idx]
        bot_label.configure(image=image)
        bot_label.image = image
        current_step += 1
        if current_step <= bot_move_number:
```



```

        step_delay = total_delay // bot_move_number
        root.after(step_delay, step)
    else:
        # Анимация выбора бота закончилась
        if on_finish:
            root.after(0, on_finish)

    step()
    return bot_chose

```

Тогда в `on_click`:

```

bot_chose_animation(
    bot_label,
    on_finish=lambda: attack(
        choosed,
        on_finish=lambda: attack(
            bot_label,
            on_finish=lambda: explosion_animation(explosion, root)
        )
    )
)

```

Краткий план того, что нужно сделать

1. Добавить в `attack` параметр `on_finish` и вызывать его в конце анимации.
2. То же желательно сделать в `explosion_animation` и (опционально) в `bot_chose_animation`.
3. В `on_click` не использовать абсолютный `root.after` по времени, а вместо этого:
 - запускать анимации через `on_finish = функции-обёртки`, которые запускают следующую анимацию.
4. Для удобства можно сделать одну функцию `run_round_animation(player_item, bot_label)`.

Если хочешь, пришли плюсом `main.py` (или хотя бы сигнатуры `bot_turn`, `move_result`, `possible_move`), и я покажу готовый `on_click` с нормальной цепочкой анимаций и примерами вызова.